Development of Git version controller with Rust

Arnau Muñoz

March 5, 2026

Resum – El desenvolupament, de "**Git version controller with Rust**", té com a objectiu el disseny i desenvolupament d'un servei de control de versions i gestió de repositoris, amb la idea d'aportar una alternativa independent a plataformes de tercers, sent una proposta gratuïta i de lliure accés. Específicament, es busca una implementació de l'aplicació amb Rust, llenguatge de programació que mitjançant el seu model de "**ownership**" i el sistema de tipus, ens permet un control exhaustiu i segur de la memòria del programa, generant així robustesa i un rendiment excepcional.

Aquest software busca la centralització de la gestió de versions, assegurant i facilitant la traçabilitat dels canvis en un context fiable i col·laboratiu pel treball distribuït en equip.

Paraules clau – Control de versions, Repositoris, Sistemes distribuïts, Desenvolupament col·laboratiu, Gestió de projectes de Software

Abstract – The development, of "**Git version controller with Rust**", aims to design and develop a version control and repository management service, with the idea of providing an independent alternative to third-party platforms, being a free and open access proposal. Specifically, we are looking for the application implementation to be done in Rust, a programming language that through its ownership model and type system, allows us exhaustive and secure memory control, generating robustness and exceptional performance.

This software seeks the centralization of version management, ensuring and facilitating the traceability of changes with a reliable and collaborative environment for distributed teamwork.

Keywords – Version control, Repositories, Distributed systems, Collaborative development, Software project management

1 Introduction - Project context

The project "**Development of Git version controller**" is framed within the current context of distributed software development and the growing need to build robust and scalable systems adapted to collaborative environments. In a technological landscape where established platforms such as "**GitHub**" or "**Bitbucket**" dominate and are the main choices of an ecosystem of version control and team coordination, the motivation arises to gain an in-depth understanding of the internal mechanisms that make such systems possible.

Beyond replicating an existing solution, this project is driven by a dual motivation. On the one hand, there is a personal motivation to develop a system more specifically tailored for personal use cases, offering greater control over its behaviour, architectural design and future grow. On the other hand, there exists interest in deconstructing and rebuilding from scratch the fundamental principles that enable globally scaled system to coordinate workspaces, manage concurrent versions and synchronize distributed repositories with multiple collaborators.

Developing this implementation allows me to adopt a critical perspective on architectural decisions, storage models, synchronisation protocols, and user experience design. This process involves not only solving technical challenges but also balancing performance, simplicity, maintainability and a decision criteria, shaping a solution aligned with the objectives.

In summary, although this Final Degree Project is based on a concept widely established within the industry, it rein-

• E-mail of contact: arnaumunozbarrera@gmail.com
• Specialty: Enginyeria del Software
• Tutorized work by: Jordi Casas Roma (Ciències de la Computació)
• Course 2025/26

terprets it from an experimental and educational perspective. The objective is not to compete with consolidated platforms, but rather achieve a deep understanding of the systems that underpin modern collaborative work and to demonstrate the feasibility of a custom made solution made from scratch.

2 OBJECTIVES

2.1 General

OB0 **Development of a version control software**: in order to gain a deeper understanding of how coordination systems operate and manage simultaneous, collaborative work across shared workspaces, with the aim of offering a more tailored alternative for specific personal use cases.

2.2 Specific

Each specific objective presented in the following section is essential and directly contributes to the validation and fulfilment of the overall purpose of the project.

OB1 **Develop a functional system:** a software system that enables the execution of version-control operations in collaborative environments, both in local and remote workspaces.

OB1 **Develop a functional system:** a software system that enables the execution of version-control operations in collaborative environments, both in local and remote workspaces.

OB2 **Implement a modular and scalable architecture:** create an architecture that allows future extension and development of the system with new functionalities, ensuring the long-term growth of the tool through an alternative and open-development approach.

OB3 **Develop an analytical interface:** design and build a system focused on analytically visualising repository status through dashboards and visual elements, enabling an intuitive understanding between the user and the system.

OB4 **Provide relevant metrics:** provide the user with relevant metrics and indicative, dynamic visualisations that improve both capability and ease of decision-making regarding project evolution and repository activity.

OB5 **Explore the viability of an alternative solution:** create a proprietary implementation as an alternative to traditional version-control systems, from a modern technical and architectural perspective, for a more user-adapted experience.

3 METHODOLOGY

For the development of this Project I implemented the "**DevOps**" methodology, it focuses on being constant with the integration process, through the automation of repetitive tasks and switching ,as needed, between the phases of development and operation, staying always on the same path to ensure quality and consistent deliverables.

The work strategy was structured into short iterative cycles, combining incremental development practices with continuous integration and validation processes. It was accomplished by following this main actions:

- **Planning and objective definition:** at the start of each cycle, the specified functionalities were decomposed, described and re-organized according to their priority, based on their impact and implementation complexity. This planning ensured a global view of the whole project, adding clear vision and supported coherent progression.

- **Development and Continuous Integration (CI):** each major code change was integrated and verified on a frequent basis, ensuring system stability and reducing the risk of conflicts or accumulated errors. This process enabled early issue detection.

- **Continuous Delivery (CD) and Deploy:** each feature development passed automated build and validation processes to ensure they could be deployed in a controlled and repeatable manner. This approach allowed the system to remain in a functional state at all times.

- **Monitoring and tracking:** project status was continuously monitored with progress and issues alerts. This ongoing oversight facilitated rapid adaptation to changes or new requirements.

Through the application of "**DevOps**" practices, the project evolved progressively and was always supported by constant feedback cycles and clear traceability across each development phase. This approach not only improved implementation deadlines but also enhanced the overall quality of the final product and its adaptability to future changes.

As shown in Figure 1 in the "Appendix", the project timeline illustrates the general structure of the tasks, as well as their temporal distribution in the Gantt chart, where the overall project planning and the achievement of the main milestones can be observed.

To deep into the process structure, the project is divided intro three main work blocks:

- BLK1 **Backend development**, focused on implementing the version control system, managing local and remote repositories and exposing a functional API.

- BLK2 **Frontend development**, oriented towards creating a minimalistic visual interface that enable the analysis and monitoring of repositories status via dashboard and relevant metrics.

- BLK3 **Transversal and documentation block**, dedicated to planning, validation, testing and results analysis

In addition, a progressive development roadmap is established to structure the system's evolution.

- **Phase 0:** Market analysis and review of existing solutions.

- **Phase 1:** Architectural design and definition of the conceptual model.
- **Phase 2:** Development of a Minimum Viable Product (MVP).
- **Phase 3:** Optimization of processes, performance and robustness.
- **Phase 4:** Validation, final testing and system evaluation.

USE OF GENERATIVE AI

In this work, the generative artificial intelligence tool, "**ChatGPT-o4 mini**", has been used for translation support. The use of this tool has been applied in the following parts of the work: summary and methodology.

All content generated with the support of AI tools has been reviewed, verified, and edited by the author prior to its inclusion in the final version of the document.

The final content is a faithful representation of the work carried out by the author and of their intellectual contributions.

The author declares under their responsibility that:

· The content of the work is truthful, complete, and accurate;

· They have made transparent use of generative AI tools;

· And they assume full responsibility for the authorship and content of this work.

REFERENCES

[1] Jira "Creador de diagramas de Gantt", [Online] Available at: https://support.atlassian.com/jira-product-discovery/docs/create-a-timeline-view/ [Revised: 20-feb-2026]

[2] DevOps "DevOps", [Online] Available at: https://es.wikipedia.org/wiki/DevOps [Revised: 26-feb-2026]

APPENDIX

A.1 Methodology

This section presents project planning designed to track deadlines, tasks, and deliverables using "**Jira**" as a management tool. The objective is to ensure effective coordination, monitoring, and control throughout the project lifecycle.

The following images provide an overview of the overall organisation of tasks. They illustrate the main development areas (Backend, Frontend and Documentation), each structured with their respective timelines, milestones and deadlines, represented through a Gantt chart.

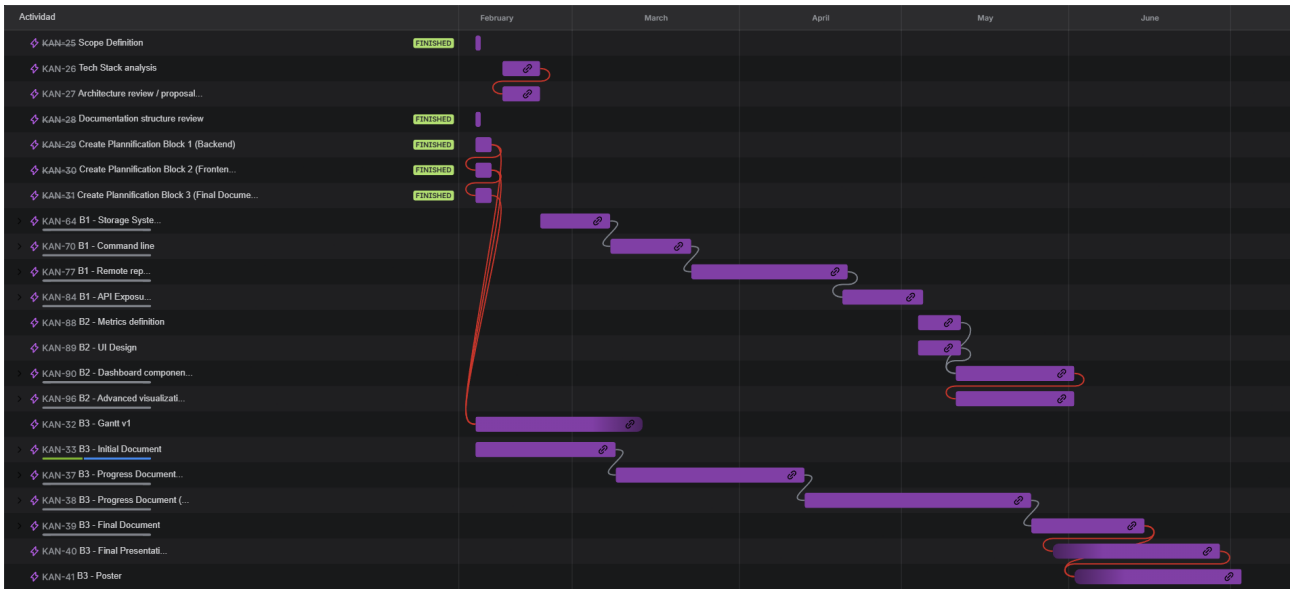

Fig. 1: Gantt diagram.

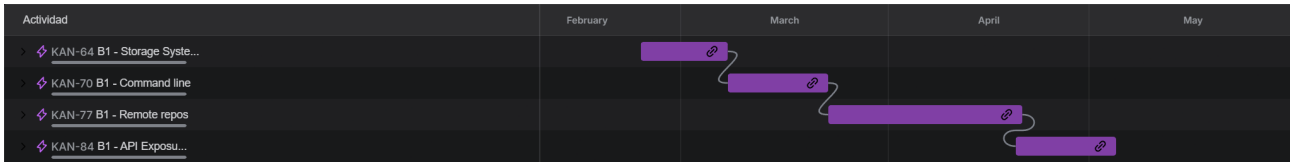

Fig. 2: B1 Backend.

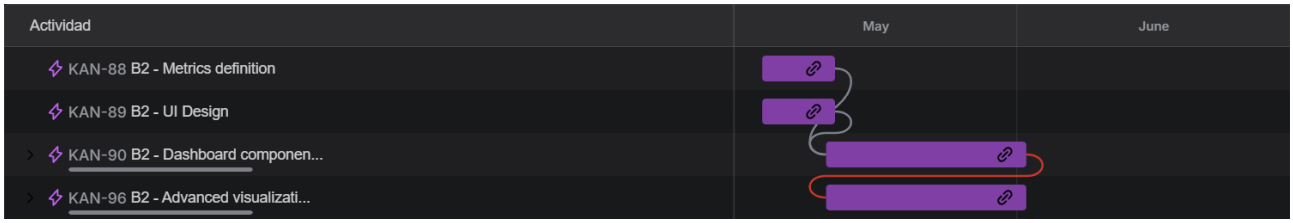

Fig. 3: B2 Frontend.

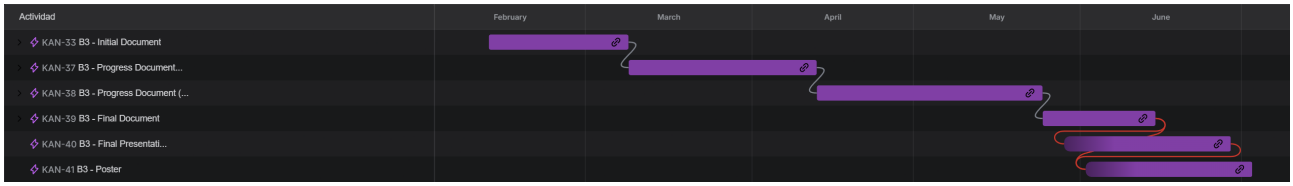

Fig. 4: B3 Documentation.